

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Омский государственный технический университет»

Кафедра «Прикладная математика и фундаментальная информатика»

ОТЧЁТ

по учебной практике

«ИТОГОВЫЙ ОТЧЁТ»

Выполнил(а): студент(ка) группы

Проверил: _____

Омск, 2026

СОДЕРЖАНИЕ

Введение	4
1 Рефлексивный анализ выполненных заданий	5
1.1 Написание функции-расширения	5
1.2 Введение в LINQ	5
1.3 Итераторы в C#	6
1.4 Интерфейсы в C#	7
1.5 Изучение класса с помощью рефлексии	8
2 Перечень и назначение модульных тестов	9
2.1 Тесты задания «Написание функции-расширения»	9
2.2 Тесты задания «Введение в LINQ»	10
2.3 Тесты задания «Итераторы в C#»	10
2.4 Тесты задания «Интерфейсы в C#»	11
2.5 Тесты задания «Изучение класса с помощью рефлексии» . . .	12
3 Анализ роли модульного тестирования	14
3.1 Задачи, решаемые модульными тестами	14
3.2 Влияние разработки через тесты на скорость работы	15
3.3 Основания идеи модульных тестов	16
3.4 Ограничения выполненного тестирования	17
4 Второй промежуточный отчёт	18
4.1 Атрибуты классов и рефлексия в C#	18
4.2 Динамические библиотеки	20
4.3 Метаданные библиотеки классов	21
4.4 Система плагинов	22
4.5 Генерация классов во время выполнения программы	23
4.6 Задачи генерации во время компиляции и выполнения	24
4.7 Статическая и динамическая компоновка	24
4.8 Влияние системы плагинов на архитектуру	25
4.9 Риски безопасности и способы их снижения	26
4.10 Примеры систем плагинов	27

4.11	Итоги рефлексивного анализа второй недели	28
5	Третий промежуточный отчёт	29
5.1	Сериализация и десериализация объектов	29
5.2	Знакомство с потоками	30
5.3	Определение оптимальных параметров алгоритма	31
5.4	Описание и интерпретация результатов эксперимента	32
5.5	Свойства задачи, допускающие многопоточное решение . . .	35
6	Итоговый отчёт	36
6.1	Разработка многопоточного сервера команд	36
6.2	Реализация планировщика команд	38
6.3	Реализация длительных команд	39
6.4	Описание и интерпретация результатов экспериментов	40
	Список использованных источников	44

ВВЕДЕНИЕ

Настоящий отчёт содержит результаты рефлексивного анализа заданий, выполненных в течение учебной практики по программированию на языке C#. Цель отчёта состоит в систематизации выполненных действий, выявлении возникших затруднений и описании способов их преодоления. Отдельное внимание уделено модульному тестированию, многопоточному выполнению команд и экспериментальной проверке принятых решений.

В ходе работы были последовательно изучены методы расширения, LINQ, итераторы, интерфейсы, рефлексия, динамическая загрузка библиотек, генерация классов, сериализация, потоки и организация очереди команд. Для заданий созданы библиотечные, консольные и тестовые проекты. Автоматическая проверка настроена с помощью GitHub Actions. На момент подготовки итогового отчёта общее решение содержит 134 успешно выполняемых модульных теста.

Таблица 1: Состав выполненных работ первой недели

№	Задание	Тестов	Результат CI
1	Написание функции-расширения	7	успешно
2	Введение в LINQ	5	успешно
3	Итераторы в C#	5	успешно
4	Интерфейсы в C#	8	успешно
5	Изучение класса с помощью рефлексии	7	успешно

1 РЕФЛЕКСИВНЫЙ АНАЛИЗ ВЫПОЛНЕННЫХ ЗАДАНИЙ

1.1 Написание функции-расширения

В первом задании была подготовлена ветка `task01`, создано решение `practice2025`, а также библиотека классов `task01` и тестовый проект `task01tests`. В статическом классе `StringExtensions` реализован метод расширения `IsPalindrome`. Для его вызова в форме метода экземпляра первый параметр объявлен как `this string input`. Обработка строки состоит из удаления пробельных символов и знаков препинания, приведения символов к единому регистру и сравнения нормализованной последовательности с её обратным представлением. После реализации были подготовлены тесты `xUnit` и настроена автоматическая сборка.

Основное затруднение было связано с определением поведения для граничных входных данных. Формального сравнения исходной строки с её перевёрнутой копией недостаточно, поскольку фразы-палиндромы содержат пробелы, знаки препинания и буквы разного регистра. Дополнительно требовалось определить результат для пустой строки, строки из пробелов и строки, состоящей только из пунктуации.

Проблема была решена посредством предварительной нормализации данных. Для классификации символов использованы `char.IsWhiteSpace` и `char.IsPunctuation`; после очистки проверяется наличие хотя бы одного значимого символа. Набор тестов был расширен граничными случаями. Такой подход позволил отделить подготовку входных данных от собственно проверки симметричности.

Наиболее сложным моментом оказалось формирование однозначного контракта метода для строк, которые после очистки становятся пустыми. Принятое решение возвращать `false` исключает формально истинный, но содержательно бессмысленный результат для пустой последовательности.

1.2 Введение в LINQ

Во втором задании создана ветка `task02`, проекты `task02` и `task02tests`, а также ссылка тестового проекта на библиотеку. Модель `Student` содержит имя, факультет и список оценок. Класс `StudentService` реализует выбор студентов факультета, фильтрацию по среднему баллу, сортировку по имени, группировку по факультету и поиск факультета с максимальным

средним баллом. Все операции с коллекциями выражены посредством LINQ.

Затруднение состояло в правильном выборе уровня агрегации. Средний балл отдельного студента вычисляется по его оценкам, тогда как средний балл факультета должен учитывать группу студентов. Ошибка в последовательности операций `GroupBy`, `Average` и `OrderByDescending` могла привести к сравнению несопоставимых величин. Отдельного внимания потребовало требование не использовать циклы.

Для решения задачи каждый метод был представлен как последовательность преобразований: фильтрация выполняется через `Where`, сортировка — через `OrderBy`, группировка — через `ToLookup`, а определение лучшего факультета — через группировку, вычисление среднего значения и выбор первого элемента после сортировки по убыванию. Корректность каждого преобразования подтверждена отдельным тестом.

Наиболее сложным оказался метод поиска факультета с максимальным средним баллом, поскольку он объединяет группировку, вложенную агрегацию и выбор итогового ключа. Его декомпозиция в последовательность LINQ-операций сделала логику прозрачной и устранила необходимость в промежуточных циклах.

1.3 Итераторы в C#

В третьем задании создана обобщённая коллекция `CustomCollection<T>`, реализующая `IEnumerable<T>`. Внутреннее хранение организовано с помощью `List<T>`. Добавлены методы `Add` и `Remove`, стандартный перечислитель, обратный итератор `GetReverseEnumerator`, генератор `GenerateSequence`, а также LINQ-метод `FilterAndSort`. Проекты `task03` и `task03tests` включены в общее решение.

Основное затруднение было связано с различием между готовой коллекцией и лениво вычисляемой последовательностью. Метод с `yield return` не формирует полный результат при вызове, а сохраняет состояние выполнения и выдаёт элементы по мере перечисления. Для обратного обхода требовалось корректно определить начальный индекс и условие завершения, особенно для пустой коллекции.

Обратный обход реализован от индекса `Count - 1` до нуля. Генератор числовой последовательности выдаёт ровно `count` значений, начиная

с `start`. Фильтрация и сортировка делегированы стандартным операциям `Where` и `OrderBy`. Тесты материализуют ленивые последовательности методом `ToList`, после чего сравнивают фактический порядок элементов с ожидаемым.

Наиболее сложным моментом стало понимание отложенного характера выполнения итератора. Результат метода с `yield return` существует не как заранее заполненный список, а как правило последовательной выдачи значений. Это свойство особенно важно при работе с большими наборами данных и цепочками LINQ.

1.4 Интерфейсы в C#

В четвёртом задании определён интерфейс `ISpaceship`, задающий операции движения, поворота и выстрела, а также свойства скорости и мощности огня. Классы `Cruiser` и `Fighter` реализуют единый контракт. Крейсер имеет скорость 50 и мощность 100, истребитель — скорость 100 и мощность 50. Для проверки поведения добавлены свойства пройденного расстояния, текущего угла и количества выпущенных ракет с закрытыми сеттерами.

Затруднение состояло в том, что методы интерфейса имеют возвращаемый тип `void`. Простая запись сообщений в консоль позволила бы вызвать методы, однако не обеспечила бы надёжной проверки результата. Требовалось сохранить заданный интерфейс и одновременно сделать поведение наблюдаемым для модульных тестов.

Проблема решена за счёт инкапсулированного состояния конкретных классов. Метод `MoveForward` увеличивает расстояние на значение скорости, `Rotate` изменяет угол, а `Fire` увеличивает счётчик ракет. Закрытая запись свойств предотвращает произвольное изменение состояния извне. Тесты подтверждают как характеристики кораблей, так и изменения после команд.

Наиболее сложным оказалось согласование абстрактного контракта и проверяемой реализации. Интерфейс должен описывать возможности корабля, не раскрывая детали хранения состояния, тогда как тестам необходимо наблюдать последствия операций. Добавление доступных только для чтения свойств обеспечило требуемый баланс полиморфизма и инкапсуляции.

1.5 Изучение класса с помощью рефлексии

В пятом задании реализован класс `ClassAnalyzer`, принимающий объект `Type`. Средствами `System.Reflection` получаются публичные методы, имена параметров и возвращаемый тип выбранного метода, все поля, публичные свойства и сведения о наличии атрибута. Для выбора категорий членов применены флаги `BindingFlags`; обработка полученных массивов выполнена с помощью LINQ без циклов.

Главное затруднение было связано с точным выбором флагов поиска. Стандартные методы рефлексии могут включать унаследованные члены, пропускать статические элементы либо не возвращать закрытые поля. Кроме того, требование к `GetMethodParams` предусматривало не только имена параметров, но и возвращаемый тип.

Для методов использована комбинация `Public`, `Instance`, `Static` и `DeclaredOnly`. Для полей дополнительно указан `NonPublic`. Имена параметров формируются оператором `Select`, а имя возвращаемого типа присоединяется оператором `Append`. Если метод отсутствует, последовательность остаётся пустой. Атрибут определяется с помощью обобщённого метода `GetCustomAttribute<T>`.

Наиболее сложным моментом стало понимание того, что результат рефлексии зависит не только от анализируемого типа, но и от заданной области поиска. Явное указание `BindingFlags` сделало поведение анализатора предсказуемым и позволило проверить публичные, приватные, статические и экземплярные члены отдельными утверждениями.

2 ПЕРЕЧЕНЬ И НАЗНАЧЕНИЕ МОДУЛЬНЫХ ТЕСТОВ

Под предусловием далее понимается состояние тестовой среды и входных данных до вызова проверяемого метода. Постусловием является наблюдаемое состояние или возвращаемое значение после выполнения метода. Совокупность тестов покрывает основные сценарии, граничные случаи и существенные свойства реализованных классов.

2.1 Тесты задания «Написание функции-расширения»

Таблица 2: Модульные тесты метода IsPalindrome

№	Тест	Проверяемый случай	Предусловие	Постусловие
1	IsPalindrome_RussianPhrase_ReturnsTrue	Подтверждает распознавание русской фразы-палиндрома с пробелами и разным регистром.	Задана строка «А роза упала на лапу Азора».	IsPalindrome возвращает true.
2	IsPalindrome_NotPalindrome_ReturnsFalse	Проверяет отклонение обычной строки, не обладающей симметрией.	Задана строка «Hello, world!».	Метод возвращает false.
3	IsPalindrome_EmptyString_ReturnsFalse	Покрывает граничный случай пустой входной строки.	Длина входной строки равна нулю.	Метод возвращает false; исключение не возникает.
4	IsPalindrome_WithPunctuation_IgnoresPunctuation	Подтверждает исключение пунктуации и пробелов при анализе английской фразы.	Задана строка «Was it a car or a cat I saw?».	После нормализации строка распознаётся как палиндром; результат равен true.
5	IsPalindrome_DifferentLetterCase_IgnoresCase	Проверяет независимость результата от регистра символов.	Задана палиндромная последовательность с чередованием прописных и строчных букв.	Метод возвращает true.
6	IsPalindrome_OnlyWhitespace_ReturnsFalse	Покрывает строку, не содержащую значимых символов после удаления пробелов.	Вход состоит только из пробельных символов.	После очистки метод возвращает false.
7	IsPalindrome_OnlyPunctuation_ReturnsFalse	Покрывает строку, состоящую только из знаков препинания.	Во входе отсутствуют буквы и цифры.	После удаления пунктуации метод возвращает false.

2.2 Тесты задания «Введение в LINQ»

Перед каждым тестом формируется единый набор из трёх студентов: Иван и Анна обучаются на факультете ФИТ, Пётр — на факультете «Экономика». Для каждого студента заданы оценки, после чего создаётся экземпляр `StudentService`.

Таблица 3: Модульные тесты класса `StudentService`

№	Тест	Проверяемый случай	Предусловие	Постусловие
1	<code>GetStudentsByFaculty_ReturnsCorrectStudents</code>	Проверяет фильтрацию студентов по названию факультета.	Сервис содержит двух студентов ФИТ и одного студента другого факультета; передан ключ «ФИТ».	Возвращены ровно два элемента, и у каждого факультет равен «ФИТ».
2	<code>GetStudentsWithMinAverageGrade_ReturnsCorrectStudents</code>	Проверяет отбор по минимальному среднему баллу.	Для студентов заданы различные наборы оценок; передано пороговое значение среднего балла.	Результат содержит только студентов, среднее значение оценок которых не меньше порога.
3	<code>GetStudentsOrderedByName_ReturnsStudentsInAlphabeticalOrder</code>	Подтверждает сортировку по имени в алфавитном порядке.	Исходная коллекция расположена не по алфавиту.	Последовательность имён в результате соответствует возрастающему лексикографическому порядку.
4	<code>GroupStudentsByFaculty_ReturnsCorrectGroups</code>	Проверяет группировку в структуру <code>ILookup</code> .	В коллекции представлены два факультета с различным числом студентов.	Созданы группы «ФИТ» и «Экономика»; количество элементов в каждой группе соответствует исходным данным.
5	<code>GetFacultyWithHighestAverageGrade_ReturnsCorrectFaculty</code>	Проверяет агрегирование оценок по факультетам и выбор максимума.	Студенты факультета «Экономика» имеют более высокий совокупный средний балл.	Метод возвращает строку «Экономика».

2.3 Тесты задания «Итераторы в C#»

Таблица 4: Модульные тесты класса CustomCollection<T>

№	Тест	Проверяемый случай	Предусловие	Постусловие
1	CustomCollection_GetEnumerator_ReturnsAllItems	Проверяет реализацию IEnumerable<T> и прямой порядок перечисления.	В пустую коллекцию последовательно добавлены числа 1 и 2.	Цикл перечисления выдаёт последовательность {1, 2} без потери и перестановки элементов.
2	GetReverseEnumerator_ReturnsItemsInReverseOrder	Проверяет обратный итератор с yield return.	Коллекция содержит числа 1 и 2 в прямом порядке.	После материализации обратного итератора получена последовательность {2, 1}.
3	GenerateSequence_ReturnsCorrectSequence	Проверяет начало, длину и шаг генерируемой числовой последовательности.	Метод вызван со значениями start = 5 и count = 3.	Результат равен {5, 6, 7}; сформировано ровно три элемента.
4	FilterAndSort_ReturnsFilteredAndSortedItems	Проверяет совместную работу Where и OrderBy.	Коллекция содержит {3, 1, 2}; предикат допускает значения больше 1.	После фильтрации и сортировки результат равен {2, 3}.
5	Remove_RemovesItem	Проверяет удаление существующего элемента и сохранение остальных элементов.	Коллекция содержит числа 1 и 2; удаляется число 1.	Remove возвращает true, а коллекция содержит только число 2.

2.4 Тесты задания «Интерфейсы в C#»

Таблица 5: Модульные тесты кораблей, реализующих ISpaceship

№	Тест	Проверяемый случай	Предусловие	Постусловие
1	Cruiser_ShouldHaveCorrectStats	Проверяет заданные характеристики крейсера через ссылку интерфейсного типа.	Создан Cruiser, присвоенный переменной ISpaceship.	Скорость равна 50, мощность выстрела равна 100.
2	Fighter_ShouldBeFasterThanCruiser	Проверяет заявленное различие скоростей двух типов кораблей.	Созданы новые экземпляры Fighter и Cruiser.	Значение Fighter.Speed строго больше Cruiser.Speed.
3	Fighter_ShouldHaveCorrectStats	Проверяет характеристики истребителя через интерфейс.	Создан Fighter, доступный как ISpaceship.	Скорость равна 100, мощность выстрела равна 50.
4	Cruiser_ShouldHaveMoreFirePowerThanFighter	Подтверждает преимущество крейсера по мощности ракет.	Созданы экземпляры обоих классов.	Cruiser.FirePower строго больше Fighter.FirePower.

Продолжение на следующей странице

Продолжение таблицы

№	Тест	Проверяемый случай	Предусловие	Постусловие
5	MoveForward_ShouldIncreaseDistanceBySpeed	Проверяет наблюдаемый эффект команды движения.	Новый крейсер имеет нулевое пройденное расстояние.	После одного вызова MoveForward расстояние равно скорости крейсера, то есть 50.
6	Rotate_ShouldChangeRotationAngle	Проверяет изменение ориентации корабля.	Новый истребитель имеет нулевой угол; передан поворот 90 градусов.	После вызова Rotate угол равен 90 градусам.
7	Fire_ShouldIncreaseFiredRocketsCount	Проверяет накопление числа выполненных выстрелов.	Новый крейсер не выпустил ни одной ракеты.	После двух вызовов Fire счётчик равен 2.
8	CruiserAndFighter_ShouldImplementISpaceship	Проверяет соответствие обоих классов общему контракту.	Созданы экземпляры крейсера и истребителя.	Оба объекта допускают приведение к ISpaceship.

2.5 Тесты задания «Изучение класса с помощью рефлексии»

Таблица 6: Модульные тесты класса ClassAnalyzer

№	Тест	Проверяемый случай	Предусловие	Постусловие
1	GetPublicMethods_ReturnsCorrectMethods	Проверяет выбор публичных экземплярных и статических методов и исключение приватного метода.	Анализатор создан для TestClass, содержащего три публичных и один приватный метод.	Результат содержит Method, MethodWithParams, StaticMethod и не содержит PrivateMethod.
2	GetMethodParams_ReturnsParameterNamesAndReturnType	Проверяет получение имён параметров и имени возвращаемого типа.	В TestClass имеется метод с параметрами name, age, возвращающий string.	Получена последовательность {name, age, String}.
3	GetMethodParams_WhenMethodDoesNotExist_ReturnsEmptyResult	Покрывает запрос сведений о несуществующем методе.	Анализатору передано имя UnknownMethod, отсутствующее в типе.	Возвращается пустая последовательность; исключение не возникает.
4	GetAllFields_IncludesPublicAndPrivateFields	Проверяет применение флагов Public и NonPublic.	Анализируемый класс содержит PublicField и _privateField.	В результате присутствуют имена обоих полей независимо от модификатора доступа.

Продолжение на следующей странице

Продолжение таблицы

№	Тест	Проверяемый случай	Предусловие	Постусловие
5	GetProperties_ReturnsPropertyNames	Проверяет получение имени публичного свойства.	В TestClass определено свойство Property.	Последовательность свойств содержит строку Property.
6	HasAttribute_WhenAttributeExists_ReturnsTrue	Проверяет обнаружение атрибута класса.	Анализируется тип AttributedClass, отмеченный SerializableAttribute.	HasAttribute<SerializableAttribute> возвращает true.
7	HasAttribute_WhenAttributeDoesNotExist_ReturnsFalse	Проверяет отрицательный сценарий поиска атрибута.	Анализируется TestClass, не имеющий атрибута сериализации.	Метод возвращает false.

Таким образом, полный набор включает 32 теста: 27 тестов xUnit и 5 тестов NUnit. Использование NUnit во втором задании обусловлено типом созданного тестового проекта; назначение проверок при этом соответствует общему принципу модульного тестирования.

3 АНАЛИЗ РОЛИ МОДУЛЬНОГО ТЕСТИРОВАНИЯ

3.1 Задачи, решаемые модульными тестами

Модульные тесты решают задачу автоматизированной проверки небольших, логически завершённых частей программной системы. В выполненных работах единицей проверки являлся отдельный метод или небольшая группа взаимосвязанных свойств класса. Такой масштаб позволяет установить непосредственную связь между входными данными, выполненной операцией и наблюдаемым результатом.

Первая функция тестов состоит в проверке соответствия реализации требованиям. Например, тесты метода `IsPalindrome` формализуют правила игнорирования регистра, пробелов и пунктуации. Без тестов эти положения остаются только текстовым описанием; с тестами они превращаются в исполняемую спецификацию. Если реализация перестаёт удовлетворять хотя бы одному правилу, система тестирования указывает конкретный нарушенный сценарий.

Вторая функция состоит в предотвращении регрессий. Изменение внутреннего алгоритма, обновление зависимостей или добавление нового задания не должно нарушать ранее реализованное поведение. Запуск всех 32 тестов после каждой сборки обеспечивает повторную проверку заданий 01–05. Следовательно, тесты создают контролируемые условия для рефакторинга: разработчик может менять структуру кода, сохраняя внешний контракт.

Третья функция связана с локализацией дефектов. Если единый большой сценарий завершается ошибкой, поиск причины может потребовать анализа множества компонентов. Небольшой тест, посвящённый одному свойству, существенно сужает область поиска. Так, отказ теста `GetReverseEnumerator_ReturnsItemsInReverseOrder` непосредственно указывает на ошибку обратного перечисления, а не на методы добавления или фильтрации.

Четвёртая функция заключается в документировании поведения на примерах. Название теста, его подготовительная часть и утверждение показывают способ использования программного интерфейса. Это особенно заметно в заданиях с итераторами и рефлексией: тесты демонстрируют мо-

мент материализации последовательности, состав ожидаемых метаданных и реакцию на отсутствие метода.

Наконец, модульные тесты являются основой автоматической проверки в CI. Локально успешный результат сам по себе не гарантирует воспроизводимости в другой среде. GitHub Actions восстанавливает зависимости, собирает решение и запускает тесты на отдельном исполнителе. Успешное прохождение этой процедуры подтверждает, что результат не зависит от случайного состояния рабочего компьютера.

3.2 Влияние разработки через тесты на скорость работы

Разработка через тесты может как замедлять отдельный этап, так и ускорять полный цикл создания и сопровождения программы. На начальном этапе требуется сформулировать сценарии, подготовить данные и определить ожидаемые результаты. Для небольшого учебного метода написание теста иногда занимает сопоставимое с реализацией время. Поэтому при оценке только длительности первичного набора кода тестирование воспринимается как дополнительная работа.

Однако такая оценка не учитывает время последующей отладки. Формулирование теста вынуждает заранее уточнить контракт метода. В задании о палиндроме это позволило определить поведение пустых и очищаемых строк до появления скрытых ошибок. В задании о рефлексии тест параметров зафиксировал, что результат должен включать не только имена параметров, но и возвращаемый тип. Чем раньше обнаружена неоднозначность, тем дешевле её устранение.

Тесты ускоряют повторные изменения. После добавления нового проекта общее решение собиралось и проверялось полностью. Ручная проверка всех пяти заданий потребовала бы многократного ввода данных и визуального сравнения результатов. Автоматический запуск выполняет те же проверки воспроизводимо и за ограниченное время. Экономия возрастает с числом изменений и продолжительностью жизненного цикла проекта.

Поэтому обоснованным является вывод, что разработка через тесты умеренно замедляет первое написание простейшей версии, но ускоряет получение устойчивого результата. Для одноразового прототипа выигрыш может быть невелик. Для кода, который развивается, проверяется преподавателем,

переносится между средами и подвергается рефакторингу, суммарные затраты времени снижаются благодаря раннему выявлению дефектов и автоматизации регрессионной проверки.

3.3 Основания идеи модульных тестов

Идея модульного тестирования основана на декомпозиции сложной системы. Программа рассматривается как совокупность компонентов с определёнными контрактами. Если каждый компонент получает контролируемые входные данные и формирует ожидаемый результат, то вероятность корректного взаимодействия компонентов повышается. Модульный тест не доказывает отсутствие всех возможных ошибок, но проверяет существенные свойства на выбранном множестве случаев.

Вторым основанием является принцип наблюдаемости. Поведение программы должно быть выражено через результат, изменение состояния или зафиксированное взаимодействие. Именно поэтому в задании об интерфейсах методы движения и выстрела связаны с инкапсулированным состоянием. Проверка только факта отсутствия исключения была бы слабой: она не подтверждала бы, что команда действительно оказала требуемый эффект.

Третьим основанием выступает воспроизводимость эксперимента. Каждый тест задаёт исходное состояние, выполняет одно действие и сравнивает наблюдение с ожидаемым постусловием. При одинаковой версии кода результат должен быть одинаковым независимо от числа запусков. Такая организация сближает тестирование с экспериментальным методом: гипотеза о корректности конкретного свойства подвергается повторяемой проверке.

Четвёртым основанием является быстрая обратная связь. Чем меньше временной интервал между внесением изменения и обнаружением нарушения, тем проще связать дефект с его причиной. Автоматический запуск тестов при отправке изменений в репозиторий создаёт единый контрольный контур: изменение кода, сборка, проверка и получение результата CI.

Следовательно, модульное тестирование основано на сочетании декомпозиции, явного контракта, наблюдаемости, изоляции и воспроизводимой проверки. Его практическая ценность определяется не количеством утвер-

ждений само по себе, а тем, насколько выбранные сценарии отражают значимые свойства программы и возможные источники ошибок.

3.4 Ограничения выполненного тестирования

Несмотря на успешное выполнение всех тестов, покрытие не является исчерпывающим доказательством корректности. Набор ориентирован на требования заданий и наиболее важные граничные случаи. Например, метод работы с палиндромами можно дополнительно проверить на строках с цифрами и символами разных алфавитов; сервис студентов — на пустой коллекции и отсутствии оценок; генератор — на нулевом и отрицательном значении `count`; анализатор — на перегруженных методах.

Указанные случаи не отменяют ценность существующих проверок, но показывают необходимость риск-ориентированного расширения набора тестов. При дальнейшем развитии проекта новые тесты должны добавляться для каждого обнаруженного дефекта и каждого уточнённого требования. Такой подход превращает набор тестов в накапливаемую базу знаний о поведении системы.

4 ВТОРОЙ ПРОМЕЖУТОЧНЫЙ ОТЧЁТ

Второй этап учебной практики был посвящён механизмам, позволяющим программе исследовать собственную структуру, загружать внешние компоненты и формировать программный код во время выполнения. В течение недели были последовательно выполнены пять заданий: «Атрибуты классов и рефлексия в C#», «Динамические библиотеки», «Метаданные библиотеки классов», «Система плагинов» и «Генерация классов во время выполнения программы». Все решения включены в общее решение `practice2025`, размещены в отдельных ветках репозитория и проверены средствами непрерывной интеграции.

Выполненные работы образуют логически связанную последовательность. Сначала были исследованы пользовательские атрибуты как способ декларативного описания программных сущностей. Затем полученные знания применялись при динамической загрузке сборок и извлечении их метаданных. На следующем этапе эти механизмы были объединены в систему плагинов с учётом зависимостей. Завершающее задание показало обратную сторону динамического программирования: программа не только анализировала готовые типы, но и создавала новый тип из исходного текста. Согласно документации Microsoft, атрибуты добавляют к программным сущностям метаданные, доступные во время выполнения через рефлексия [6]. Это положение стало общей теоретической основой большинства заданий второй недели.

Таблица 7: Состав выполненных работ второй недели

№	Задание	Тестов	Результат CI
7	Атрибуты классов и рефлексия в C#	6	успешно
8	Динамические библиотеки	4	успешно
9	Метаданные библиотеки классов	4	успешно
10	Система плагинов	6	успешно
11	Генерация классов во время выполнения	8	успешно

4.1 Атрибуты классов и рефлексия в C#

Выполнение задания началось с определения назначения пользовательских атрибутов. Было установлено, что атрибут представляет собой декларативную характеристику типа или его члена и не изменяет поведение

программы самостоятельно. Реакция на атрибут появляется только в том компоненте, который считывает метаданные и интерпретирует их. На основании этого положения были созданы классы `DisplayNameAttribute` и `VersionAttribute`. Для каждого атрибута была задана допустимая область применения. Атрибут отображаемого имени применялся к классам, методам и свойствам, а атрибут версии — к классу.

После определения атрибутов был создан демонстрационный класс `SampleClass`. Класс получил отображаемое имя «Пример класса» и версию 1.0. Его метод и свойство также были отмечены атрибутами отображаемого имени. Затем был реализован статический класс `ReflectionHelper`. Метод `PrintTypeInfo` принимал объект `Type`, считывал атрибуты класса и перебирал его публичные методы и свойства. Результат выводился в консоль в понятном человеку виде. Корректность решения проверялась шестью модульными тестами: четыре теста подтверждали значения отдельных атрибутов, ещё два проверяли полный вывод и поведение для типа без пользовательских атрибутов.

Основная трудность состояла в разграничении стандартных и пользовательских членов типа. При получении методов через рефлекссию в результат могут попадать унаследованные методы `Object`, а свойства имеют методы доступа, которые также присутствуют в метаданных. Проблема была решена явным выбором требуемых элементов и проверкой наличия `DisplayNameAttribute` перед формированием строки вывода. Такой подход позволил не включать в результат служебную информацию, не относящуюся к заданию.

Наиболее сложным моментом стало понимание связи между декларативной записью атрибута и выполняемой логикой. Внешне атрибут напоминает специальную команду, однако фактически он является объектом метаданных. Поэтому требовалось отдельно реализовать код, который обнаруживает атрибут и использует его свойства. Данная особенность была закреплена тестом для класса без атрибутов: отсутствие метаданных не должно приводить к ошибке и не должно создавать вымышленные значения.

4.2 Динамические библиотеки

В задании о динамических библиотеках были созданы три функциональные части. Первая часть представлена библиотекой `CommandLib`, содержащей интерфейс `ICommand` с методом `Execute`. Вторая часть — библиотека `FileSystemCommands` с двумя реализациями команд. Команда `DirectorySizeCommand` вычисляет суммарный размер файлов выбранного каталога, а `FindFilesCommand` находит файлы, соответствующие переданной маске. Третьей частью стало консольное приложение `CommandRunner`, которое получает путь к библиотеке, загружает сборку, обнаруживает реализации интерфейса, создаёт их экземпляры и выполняет команды.

Для загрузки использовался метод `Assembly.LoadFrom`, предназначенный для загрузки сборки по указанному пути [7]. После загрузки выполнялся анализ типов сборки. Кандидат должен был быть конкретным классом, реализующим `ICommand`. Создание экземпляра выполнялось с учётом параметров конструктора конкретной команды. Такой подход продемонстрировал, что интерфейс служит стабильным контрактом между приложением и динамически подключаемой библиотекой.

Трудности возникли при тестировании команд, выводящих результат в консоль. Первоначальная проверка только отсутствия исключения не доказывала правильность вычисления. Для повышения точности тестов стандартный поток вывода временно перенаправлялся в `StringWriter`. После выполнения команды из полученной строки извлекался результат и сравнивался с ожидаемым значением. Для каждого теста создавался отдельный временный каталог с заранее известными файлами, а после проверки каталог удалялся. Это обеспечило независимость тестов от содержимого пользовательской файловой системы.

Наиболее сложной частью стала организация границы между основной программой и загружаемой библиотекой. Если интерфейс объявить только в приложении или продублировать в разных проектах, одинаковые по имени интерфейсы будут различными типами для среды выполнения. Решением стало вынесение контракта в отдельную библиотеку `CommandLib`, на которую ссылаются обе стороны. В результате консольное приложение может работать с неизвестными на этапе компиляции реализациями, если они со-

блюдают общий контракт.

4.3 Метаданные библиотеки классов

Задание по метаданным продолжило работу с динамическими сборками, но изменило цель анализа. Вместо поиска только команд требовалось вывести полное структурированное описание библиотеки: список классов, их атрибутов, методов и конструкторов, а также имена и типы параметров методов и конструкторов. Для решения было разработано консольное приложение, принимающее путь к DLL через аргументы командной строки.

После проверки существования файла сборка загружалась в память. Для каждого класса последовательно формировались разделы с атрибутами, конструкторами и методами. Параметры преобразовывались в текстовые записи, включающие имя типа и имя параметра. Для методов дополнительно выводился тип возвращаемого значения. Атрибуты из задания 07 были добавлены к анализируемым классам, благодаря чему программа демонстрировала не только стандартные метаданные .NET, но и пользовательские значения.

Основное затруднение было связано с объёмом данных, доступных через рефлексию. В сборке присутствуют автоматически созданные методы доступа к свойствам, унаследованные члены и служебные типы. Неконтролируемый вывод делал результат громоздким и затруднял его проверку. Проблема была решена с помощью явных условий отбора и единообразных методов форматирования. Для сохранения предсказуемого порядка элементы сортировались по имени. Это особенно важно для модульных тестов, поскольку один и тот же набор метаданных должен формировать стабильный текст на разных запусках.

Наиболее сложным моментом оказалось корректное представление сигнатур. Метод и конструктор могут не иметь параметров, содержать несколько параметров различных типов или использовать типы с пространствами имён. Было необходимо избежать неоднозначных строк и одновременно сохранить читаемость результата. Решение состояло в общем преобразовании каждого `ParameterInfo` в пару «тип — имя» и объединении таких пар в порядке следования параметров. Корректность была подтверждена тестами для классов, атрибутов, методов и конструкторов.

4.4 Система плагинов

В задании о системе плагинов были объединены интерфейсы, атрибуты, динамическая загрузка и анализ зависимостей. В качестве общего контракта использовался интерфейс `ICommand` из задания 08. Был определён атрибут `PluginLoadAttribute`, которым отмечались классы, предназначенные для автоматической загрузки. Вторым атрибутом, `PluginDependencyAttribute`, хранил тип плагина, который должен быть загружен раньше текущего.

Реализованный `PluginManager` обнаруживает DLL-файлы в указанном каталоге, загружает сборки и выбирает конкретные классы, одновременно реализующие `ICommand`, имеющие открытый конструктор без параметров и отмеченные атрибутом `PluginLoad`. Затем строится порядок загрузки. В демонстрационном наборе плагин регистратора не имеет зависимостей, хранилище зависит от регистратора, а генератор отчётов зависит от хранилища. После определения порядка создаются экземпляры и последовательно вызывается метод `Execute`.

Главная трудность была связана с тем, что простой порядок файлов в каталоге не отражает логических зависимостей. Если загружать плагины в алфавитном или случайном порядке, зависимый компонент может начать работу раньше необходимого ему компонента. Зависимости были представлены ориентированным графом. Для каждого плагина выполнялся поиск в глубину: сначала обрабатывались зависимости, после чего в результат добавлялся сам плагин. Дополнительно использовались состояния «обрабатывается» и «обработан». Повторное посещение вершины в состоянии «обрабатывается» означает цикл, при котором корректный порядок не существует.

Наиболее сложным моментом стала обработка ошибочных конфигураций. Отсутствующую зависимость нельзя игнорировать, поскольку тогда система запустит плагин в недопустимом состоянии. Циклическую зависимость также нельзя разрешить выбором произвольного порядка. Поэтому для обоих случаев формируются явные исключения с понятным описанием причины. Шесть тестов проверяют обнаружение всех плагинов, наличие атрибутов, создание экземпляров, правильный порядок выполнения и выявление цикла.

4.5 Генерация классов во время выполнения программы

Заключительное практическое задание состояло в создании класса `Calculator`, отсутствующего в скомпилированной библиотеке. Текст класса хранился в строковой константе и включал четыре метода: `Add`, `Minus`, `Mul` и `Div`. Для компиляции использовались API платформы Roslyn, предоставляющие программный доступ к синтаксической и семантической модели C# [8].

Сначала исходная строка преобразовывалась в синтаксическое дерево. Затем формировался набор ссылок на системные сборки и сборку с контрактом `ICalculator`. Объект `CSharpCompilation` создавал динамическую библиотеку в потоке памяти. При неуспешной компиляции диагностические сообщения объединялись и включались в исключение. При успешной компиляции байты сборки загружались в память, находился тип `Calculator` и создавался его экземпляр.

Условие запрещало вызывать арифметические методы через рефлексию. Проблема была решена изменением исходного определения класса: динамический `Calculator` реализует заранее известный интерфейс `ICalculator`. Рефлексия применяется только на границе создания экземпляра, поскольку имя нового типа неизвестно скомпилированному коду. После приведения объекта к интерфейсу выражения `calculator.Add(a, b)` и аналогичные вызовы являются обычными виртуальными вызовами через контракт и не используют `MethodInfo.Invoke`.

Трудность заключалась в формировании полного набора ссылок компиляции. Наличие типа `object` не гарантирует доступность всех системных типов, используемых компилятором. Для решения использовался список доверенных платформенных сборок текущей среды выполнения, к которому добавлялась сборка с интерфейсом `ICalculator`. Наиболее сложным оказался переход между двумя мирами типов: статически известным интерфейсом и типом, появившимся только после компиляции строки. Восемь тестовых методов проверили арифметические операции, деление на ноль, создание экземпляра, использование переданной строки и обработку синтаксической ошибки.

4.6 Задачи генерации во время компиляции и выполнения

Генерация кода во время компиляции решает задачи устранения повторяющегося шаблонного кода, построения адаптеров, сериализаторов, регистрационных таблиц и иных структур, которые могут быть полностью определены по исходной программе. Результат такой генерации становится частью итоговой сборки. Компилятор проверяет типы, а разработчик получает ошибки до запуска приложения. Подход целесообразен, когда входные данные известны заранее и редко изменяются во время работы системы. Примерами являются генерация клиентов по описанию API, реализация преобразований данных и автоматическое создание вспомогательных методов для моделей.

Генерация во время выполнения применяется тогда, когда исходная информация становится известна только после запуска. К ней относятся пользовательские формулы, сценарии, динамические правила обработки и специализированные запросы. Программа может получить текст или модель операции, скомпилировать её и выполнить без выпуска новой версии основного приложения. Такой подход повышает гибкость, но переносит часть ошибок с этапа сборки на этап выполнения и требует дополнительных мер контроля.

Различие между подходами определяется моментом фиксации результата. Компиляционная генерация увеличивает объём работы сборочной системы, зато созданный код может быть заранее проанализирован и протестирован. Генерация во время выполнения допускает адаптацию к текущим данным и настройкам, но потребляет время и память при запуске, усложняет диагностику и может создавать риск исполнения непроверенного кода. Следовательно, выбор должен основываться не на максимальной динамичности, а на том, действительно ли изменяемость после поставки является требованием системы.

4.7 Статическая и динамическая компоновка

При статической компоновке состав зависимостей определяется во время сборки. Проект содержит ссылки на библиотеки, компилятор проверяет доступность типов и методов, а необходимые компоненты поставляются как заранее определённый набор. Изменение контракта обычно выявляется ошибкой компиляции. Такой подход обеспечивает высокую предсказуемость,

упрощает навигацию по коду и позволяет инструментам разработки выполнять полный статический анализ.

Динамическая компоновка переносит выбор конкретной реализации на время выполнения. Основное приложение может знать только общий интерфейс, а реализация обнаруживается по пути к DLL, конфигурации или атрибуту. Это позволяет добавлять компоненты без повторной компиляции ядра, выбирать разные реализации для разных условий и уменьшать прямую связанность модулей. На этой основе строятся расширяемые редакторы, серверные платформы и системы обработки данных.

Преимуществами статической компоновки являются раннее обнаружение ошибок, простая модель развёртывания и меньшая неопределённость поведения. Недостатком является необходимость пересборки и повторной поставки приложения при изменении набора компонентов. Динамическая компоновка обеспечивает расширяемость и независимое обновление модулей, но создаёт риск несовместимости версий, отсутствия файла, ошибки загрузки и нарушения порядка инициализации. Кроме того, часть связей становится невидимой для компилятора и должна проверяться собственным кодом и интеграционными тестами.

Рациональная архитектура часто сочетает оба подхода. Базовые функции и контракты связываются статически, поскольку они определяют устойчивое ядро приложения. Дополнительные возможности подключаются динамически через узкие интерфейсы. В выполненной работе статической частью являлся ICommand, а плагины регистратора, хранилища и генератора отчётов представляли динамические реализации.

4.8 Влияние системы плагинов на архитектуру

Система плагинов переводит архитектуру от монолитного набора функций к модели «ядро — расширения». Ядро отвечает за жизненный цикл, обнаружение, проверку и запуск компонентов. Плагин реализует ограниченный контракт и не должен зависеть от внутренних деталей приложения. Такое разделение уменьшает необходимость изменять ядро при добавлении новых возможностей и поддерживает принцип открытости для расширения.

Одновременно архитектура объективно усложняется. Появляются пра-

вила совместимости, каталогизация расширений, обработка ошибок загрузки, управление версиями и порядок инициализации. При наличии зависимостей требуется граф, а не простой список. Становится сложнее выполнять сквозную отладку, поскольку часть кода может быть поставлена независимо от основной программы. Поэтому сама по себе система плагинов не является упрощением; она обменивает простоту небольшой программы на управляемую расширяемость большой системы.

Усложнение оправдано, если приложение действительно должно поддерживать независимые команды, интеграции или функциональные модули. Для небольшой программы с фиксированным набором операций плагины создадут избыточную инфраструктуру. Для редактора, среды разработки или корпоративной платформы преимущества независимого развития компонентов обычно превышают расходы на механизм загрузки. Важным условием является минимальный и стабильный контракт: чем больше внутреннего состояния ядра доступно плагину, тем сильнее связанность и тем труднее обновление.

4.9 Риски безопасности и способы их снижения

Основной риск плагинной системы связан с тем, что динамически загруженная библиотека является исполняемым кодом. Получив те же права, что и основной процесс, вредоносный плагин может читать и изменять файлы пользователя, отправлять данные по сети, запускать процессы или нарушать работу приложения. Цифровая подпись файла сама по себе не делает код безопасным, однако позволяет проверить издателя и целостность поставки.

Второй риск состоит в подмене библиотеки. Если каталог плагинов доступен для записи постороннему процессу или путь задаётся без проверки, приложение может загрузить другой файл с подходящим именем. Третий риск связан с уязвимостями обычного добросовестного плагина: ошибка обработки входных данных, чрезмерное потребление памяти или бесконечное выполнение способны привести к отказу всей программы. Дополнительную опасность создают несовместимые зависимости и загрузка нескольких версий одной библиотеки.

Снижение рисков требует сочетания организационных и технических мер. Следует загружать расширения только из доверенного каталога, прове-

рять издателя и контрольную сумму, ограничивать перечень разрешённых плагинов и журналировать операции загрузки. Контракт должен предоставлять минимально необходимые возможности. Потенциально опасные расширения целесообразно запускать в отдельном процессе с ограниченными правами, поскольку обычная загрузка DLL внутрь процесса не создаёт границы безопасности. Для внешнего процесса можно установить ограничения времени, памяти и доступа к ресурсам, а взаимодействие организовать через проверяемый протокол.

Необходимо также обрабатывать исключения на границе плагина, проверять совместимость версий и не передавать расширению секретные данные без необходимости. При динамической генерации кода аналогичные риски возникают при компиляции текста, поступившего от пользователя. Запуск такого кода в основном процессе допустим только для полностью доверенного источника. В противном случае требуется изоляция или отказ от произвольного языка в пользу ограниченной предметно-ориентированной модели.

4.10 Примеры систем плагинов

С системами плагинов пользователь регулярно сталкивается в веб-браузерах. Расширения могут изменять страницы, блокировать рекламу, управлять паролями и добавлять средства разработчика. Браузер предоставляет им формализованный API и запрашивает разрешения на доступ к вкладкам, сайтам и данным. Этот пример показывает одновременно преимущества расширяемости и необходимость модели безопасности.

Другим примером являются среды разработки Visual Studio и Visual Studio Code. Расширения добавляют поддержку языков, форматирование, анализ кода, интеграцию с системами контроля версий и пользовательские команды. Ядро среды не обязано содержать реализацию каждого языка, поскольку функциональность развивается независимыми поставщиками. Аналогичная модель используется в графических редакторах и программах трёхмерного моделирования, где плагины реализуют форматы файлов, фильтры и инструменты обработки.

Системы управления содержимым также применяют плагины. В WordPress расширения добавляют формы, электронную коммерцию, оптимизацию и интеграции. Игровые проекты используют модификации для

новых объектов, сценариев и правил. Медиапроигрыватели подключают кодеки и модули вывода. Во всех случаях сохраняется общий принцип: ядро определяет точки расширения, а внешние компоненты реализуют договорённое поведение.

Практические задания показали внутреннее устройство подобных решений в уменьшенном масштабе. Атрибут `PluginLoad` выполнял роль декларации расширения, интерфейс `ICommand` задавал точку взаимодействия, рефлексия обеспечивала обнаружение, а граф зависимостей определял порядок запуска. Поэтому знакомые пользовательские системы перестали восприниматься как единый неразделимый продукт: их функциональность может быть результатом совместной работы ядра и множества независимо загружаемых модулей.

4.11 Итоги рефлексивного анализа второй недели

Работы второй недели продемонстрировали последовательный переход от статически определённой структуры программы к управляемой динамичности. Атрибуты позволили описывать намерение, рефлексия — считывать это описание, динамическая загрузка — подключать неизвестные заранее сборки, система плагинов — организовывать их совместную работу, а `Roslyn` — создавать новый тип из исходного текста.

Наиболее значимые трудности были связаны с границами между компонентами. Для динамических библиотек такой границей являлся общий интерфейс, для метаданных — правила отбора и форматирования, для плагинов — граф зависимостей, а для динамической генерации — интерфейс между статически скомпилированным кодом и новым типом. Во всех случаях устойчивое решение достигалось за счёт явного контракта, проверки ошибочных состояний и модульных тестов.

Полученные результаты подтверждают, что динамические механизмы не отменяют требования строгой структуры. Напротив, чем позже определяется конкретная реализация, тем точнее должны быть заданы контракты, правила совместимости и меры безопасности. Следовательно, профессиональное применение рефлексии, плагинов и генерации кода требует не только знания API, но и архитектурного анализа последствий каждого динамического решения.

5 ТРЕТИЙ ПРОМЕЖУТОЧНЫЙ ОТЧЁТ

Третий этап учебной практики был посвящён сериализации объектов, организации параллельных вычислений и экспериментальному определению параметров многопоточного алгоритма. Работы выполнялись последовательно: сначала был изучен способ сохранения структурированных данных в формате JSON, затем реализован параллельный метод трапеций, после чего проведено измерение его точности и производительности. Во всех заданиях использовались модульные тесты и автоматическая проверка проекта.

Таблица 8: Состав работ третьей недели

№	Задание	Тестов	Результат
13	Сериализация и десериализация объектов в JSON	14	успешно
14	Знакомство с потоками	17	успешно
15	Определение оптимальных параметров вычисления интеграла	12	успешно

5.1 Сериализация и десериализация объектов

В задании были определены классы `Student` и `Subject`. Объект студента содержит имя, фамилию, дату рождения и список предметов с оценками. Для преобразования объектов применён стандартный механизм `System.Text.Json` [9]. Работа была выполнена в несколько этапов. Сначала создана модель данных, затем разработан сервис сериализации, после этого добавлены сохранение и загрузка JSON-файла и, наконец, написаны тесты для корректных и ошибочных входных данных.

Параметры сериализатора настроены в одном месте. Свойства со значением `null` не включаются в итоговый JSON, поэтому файл не содержит лишних полей. Для даты рождения создан пользовательский конвертер, использующий формат `dd.MM.yyyy`. Такой подход обеспечивает одинаковое представление даты независимо от региональных параметров операционной системы. При чтении конвертер принимает только ожидаемый формат и сообщает об ошибке, если строка не является корректной датой.

После десериализации выполняется проверка объекта. Имя и фамилия не должны быть пустыми, дата рождения не может находиться в будущем,

название предмета должно быть задано, а оценка должна входить в допустимый диапазон от 1 до 5. Проверка после преобразования необходима потому, что синтаксически правильный JSON ещё не гарантирует предметную корректность данных. Например, значение оценки 20 может быть успешно прочитано как целое число, но не может считаться допустимой оценкой студента.

При выполнении задания возникло несколько затруднений. Первое было связано с тем, что стандартный кодировщик JSON может заменять кириллические символы последовательностями Unicode. Это не нарушает корректность файла, но снижает его читаемость. Проблема была решена настройкой кодировщика, сохраняющего текст в понятном виде. Второе затруднение состояло в разграничении ошибок формата и ошибок предметной области. Неверная дата должна приводить к ошибке преобразования, а пустое имя или недопустимая оценка — к ошибке валидации. Для этого ответственность была разделена между конвертером и отдельным методом проверки данных.

Наиболее сложным моментом оказалось обеспечение симметрии операций: объект после последовательности «сериализация — десериализация» должен сохранять все значимые данные. Одновременно требовалось игнорировать отсутствующие значения, использовать специальный формат даты и отклонять недопустимые объекты. Корректность решения подтверждена тестами на обычную сериализацию, отсутствие полей со значением `null`, формат даты, сохранение и загрузку файла, повреждённый JSON, будущую дату рождения и оценки вне установленного диапазона.

5.2 Знакомство с потоками

Во втором задании недели реализовано вычисление определённого интеграла методом трапеций. Для соседних точек x_i и x_{i+1} площадь элементарной трапеции вычисляется по формуле

$$S_i = \frac{f(x_i) + f(x_{i+1})}{2} (x_{i+1} - x_i).$$

Общий результат представляет собой сумму площадей всех трапеций. Благодаря аддитивности интеграла исходный отрезок был разделён на части по количеству потоков, а каждая часть вычислялась независимо.

Для каждой части создавался объект `Thread` [10]. Основной поток

ожидал завершения рабочих потоков при помощи примитива `Barrier` [11], а затем дополнительно выполнял соединение потоков, чтобы гарантировать освобождение ресурсов. Локальный результат каждого рабочего потока добавлялся к общей сумме атомарно. Поскольку для типа `double` отсутствует простая операция атомарного сложения, использован цикл на основе `Interlocked.CompareExchange` [12]. Каждый поток сначала вычисляет результат своего отрезка в локальной переменной и обращается к общей переменной только один раз. Это уменьшает количество синхронизаций.

Особое внимание уделено границам разбиения. Части отрезка не должны пересекаться и не должны оставлять пропуски. Последняя трапеция может иметь ширину меньше заданного шага, поэтому её правая граница ограничивается значением b . Также предусмотрены проверки входных данных: функция не может быть равна `null`, шаг и количество потоков должны быть положительными, а левая граница не должна превышать правую.

Основная трудность состояла в безопасном суммировании результатов. Обычная операция `total+=localResult` включает чтение, сложение и запись, поэтому два потока могут потерять результат друг друга. Атомарное обновление устранило данную гонку. Дополнительная сложность возникла при обработке исключений внутри рабочих потоков: исключение нельзя просто передать в основной поток. Для этого ошибки собираются в потокобезопасной очереди, а после завершения вычислений основная функция формирует общее исключение.

Наиболее сложным моментом стала согласованная работа барьера, завершения потоков и обработки ошибок. Каждый рабочий поток должен обязательно достигнуть точки синхронизации даже при возникновении исключения, иначе основная функция будет ожидать бесконечно. Поэтому сигнал барьеру размещён в гарантированно выполняемой части кода. Тесты проверяют функции x и $\sin(x)$, разные количества потоков, обратимые симметричные интервалы, нулевую длину отрезка и некорректные параметры.

5.3 Определение оптимальных параметров алгоритма

Эксперимент проводился для функции $\sin(x)$ на отрезке $[-100; 100]$ при требуемой точности 10^{-4} . Теоретическое значение интеграла равно нулю, так как функция нечётная, а отрезок симметричен. Для корректного сравнения

дополнительно реализован метод `SolveSingleThread`, который выполняет тот же метод трапеций без создания объектов `Thread`, барьера и атомарных операций.

Время измерялось классом `Stopwatch` [13]. Каждый вариант запускался три раза, после чего вычислялось среднее арифметическое. Исследованы шаги 10^{-1} , 10^{-2} , 10^{-3} , 10^{-4} , 10^{-5} и 10^{-6} . Все они обеспечили требуемую точность. Абсолютные ошибки находились приблизительно в диапазоне от $2,58 \cdot 10^{-15}$ до $5,02 \cdot 10^{-13}$. Из заданного набора был выбран минимальный размер шага 10^{-6} . Он создаёт наибольшее количество трапеций и тем самым позволяет объективнее оценить выгоду параллельного вычисления.

Для выбранного шага программно исследованы варианты от 1 до 16 потоков. Число 16 соответствует количеству логических процессоров использованной системы. Результаты каждого запуска сохранены в CSV-файлы, а графики построены библиотекой `ScottPlot` [14]. Минимальное среднее время 190,761 мс было получено при 15 потоках. При 16 потоках среднее время увеличилось до 194,406 мс, поэтому оптимальным для данной системы признано значение 15.

Однопоточный алгоритм без использования потоков выполнялся в среднем за 1617,999 мс. Следовательно, преимущество лучшего многопоточного варианта составило

$$\frac{1617,999 - 190,761}{1617,999} \cdot 100\% = 88,21\%.$$

Полученное ускорение значительно превышает требуемые 15 процентов. Наиболее сложным этапом исследования было обеспечить сопоставимость результатов. Однопоточная и многопоточная версии используют одну формулу и одинаковый шаг, замеры повторяются одинаковое число раз, а на график выводятся средние значения. Благодаря этому измеряется влияние организации вычислений, а не различие математических алгоритмов.

5.4 Описание и интерпретация результатов эксперимента

На рисунке 1 по оси абсцисс указано среднее время выполнения функции `Solve`, а по оси ординат — количество потоков, как требовалось в задании. При переходе от одного потока к двум время уменьшается с 1597,835 до 982,655 мс. Дальнейшее увеличение количества потоков также ускоряет

вычисление, но прирост постепенно становится меньше.

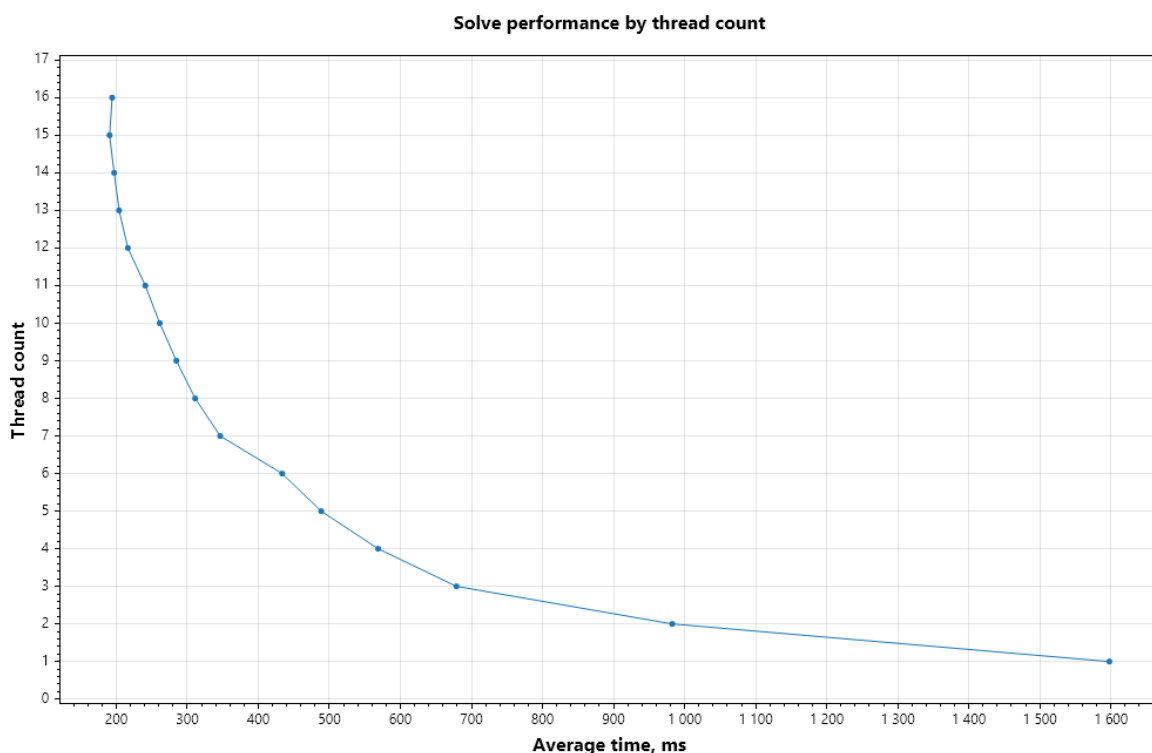


Рис. 1: Зависимость среднего времени вычисления от количества потоков

Форма графика объясняется сочетанием параллельного выполнения и накладных расходов. В начале каждый дополнительный поток получает значительную самостоятельную часть отрезка, поэтому вычислительная нагрузка распределяется между ядрами. Однако ускорение не может быть строго пропорционально числу потоков. Время расходуется на создание потоков, работу планировщика, синхронизацию и итоговое суммирование. Кроме того, потоки конкурируют за вычислительные ресурсы и общие уровни кэш-памяти. Поэтому после определённого момента накладные расходы становятся сопоставимы с выигрышем. Небольшое ухудшение результата при 16 потоках является нормальным проявлением этой зависимости и подтверждает необходимость экспериментального, а не формального выбора параметра.

Рисунок 2 показывает непосредственное сравнение реализаций. Высота столбца многопоточного варианта существенно меньше. Разница в 1427,238 мс соответствует ускорению на 88,21%. Для шага 10^{-6} на отрезке длиной 200 выполняется около 200 миллионов вычислительных шагов. При такой нагрузке полезная работа значительно превосходит затраты на запуск и синхронизацию потоков, вследствие чего параллельный вариант

оказывается эффективным.

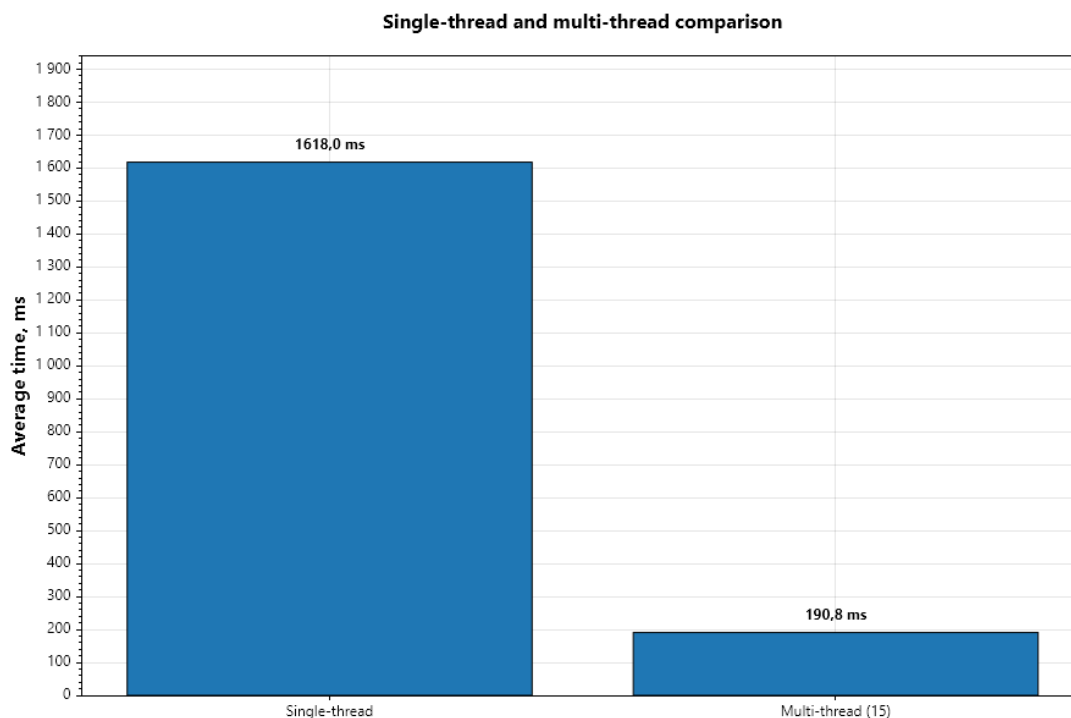


Рис. 2: Сравнение однопоточной и оптимальной многопоточной реализаций

Результаты исследования шага требуют отдельной интерпретации. Обычно уменьшение шага повышает точность метода трапеций, но в проведённом опыте даже шаг 10^{-1} дал очень малую ошибку. Причиной является специальный выбор функции и интервала: значения нечётной функции в симметричных точках взаимно компенсируются. Поэтому полученный вывод о точности нельзя автоматически переносить на произвольные функции и интервалы. В данном задании минимальный шаг 10^{-6} выбран не потому, что более крупные шаги нарушили точность, а потому, что именно он является минимальным из предложенного ряда и создаёт достаточную вычислительную нагрузку.

Усреднение трёх запусков снижает влияние фоновых процессов, изменения частоты процессора и работы планировщика, однако полностью не исключает случайные колебания. Оптимальное значение 15 относится к конкретной системе с 16 логическими процессорами и используемой версии .NET. На другом компьютере минимум графика может находиться в иной точке. Следовательно, главным результатом является не универсальное число потоков, а воспроизводимая процедура его определения.

5.5 Свойства задачи, допускающие многопоточное решение

Вычисление определённого интеграла методом трапеций хорошо поддаётся распараллеливанию прежде всего благодаря аддитивности:

$$\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx.$$

Отрезок можно разделить на любое количество непересекающихся частей, вычислить частичные интегралы независимо и сложить результаты. Рабочим потокам не требуется изменять общую структуру данных или ожидать результаты соседних потоков. Функция и параметры являются общими, но используются только для чтения, а промежуточная сумма хранится локально.

Вторым важным свойством является однородность работы. При постоянном шаге близкие по длине подотрезки содержат приблизительно одинаковое количество трапеций. Это позволяет равномерно распределить нагрузку и уменьшает вероятность того, что основной поток будет долго ожидать один перегруженный участок. Третье свойство — малый объём объединения результатов: независимо от числа трапеций каждый поток возвращает только одно число. Следовательно, объём синхронизации существенно меньше объёма вычислений.

Вместе с тем многопоточность эффективна не всегда. При крупном шаге вычислений мало, поэтому создание и синхронизация потоков могут занять больше времени, чем сам интеграл. Если вычисляемая функция изменяет общее состояние, требует последовательного доступа к ресурсу или имеет очень различную стоимость на разных участках, независимость частей нарушается либо возникает дисбаланс нагрузки. Порядок сложения чисел с плавающей точкой также может немного изменять итог вследствие конечной точности представления. Поэтому результат следует сравнивать с допустимой погрешностью, а не требовать побитового совпадения.

Таким образом, аддитивность интеграла, независимость подотрезков и компактность частичного результата создают условия для ускорения. Экспериментальные замеры позволяют найти границу, после которой увеличение числа потоков перестаёт быть полезным.

6 ИТОГОВЫЙ ОТЧЁТ

Заключительный этап учебной практики был посвящён последовательной разработке системы выполнения команд. Сначала был создан отдельный поток с очередью операций и двумя способами завершения. Затем система была дополнена планировщиком, который позволяет выполнять длительные операции небольшими частями. На последнем шаге разработанная схема была проверена на пяти экземплярах команды, каждый из которых должен был получить управление три раза. Связь между заданиями являлась принципиальной: планировщик из задания 18 использовал поток из задания 17, а длительные команды из задания 19 выполнялись средствами этого планировщика.

Таблица 9: Состав работ заключительного этапа практики

№	Задание	Тестов	Результат СИ
17	Разработка многопоточного сервера команд	10	успешно
18	Реализация планировщика команд	11	успешно
19	Реализация длительных команд	6	успешно

6.1 Разработка многопоточного сервера команд

Работа над сервером команд началась с определения минимального контракта операции. Интерфейс `ICommand` содержит единственный метод `Execute` , поэтому поток не зависит от конкретного назначения команды. После этого был создан класс `ServerThread` . В его конструкторе формируется отдельный объект `Thread` , а метод `Start` запускает цикл обработки. Новые команды передаются через потокобезопасную очередь `BlockingCollection< ICommand >` , построенную на стандартных средствах .NET [15]. Такой выбор позволил отделить добавление команд внешними потоками от их последовательного исполнения серверным потоком.

Следующим шагом стало определение правил остановки. Команда `HardStopCommand` переводит сервер в режим немедленного завершения. После её выполнения цикл прекращается, даже если в очереди остаются элементы. Команда `SoftStopCommand` , напротив, запрещает дальнейшее добавление операций, но сохраняет обработку уже принятых элементов. Поток заканчивает работу только после опустошения очереди. Для исключения оши-

бочного управления предусмотрена проверка текущего потока: обе команды остановки могут успешно выполняться только внутри того экземпляра `ServerThread`, для которого они созданы. Попытка вызвать их извне приводит к исключению.

Основное затруднение заключалось в различии между немедленным и корректным отложенным завершением. Простого логического признака остановки было недостаточно. Если устанавливать его одинаково для обоих режимов, то мягкая остановка теряет оставшиеся команды, а жёсткая, наоборот, продолжает ненужную обработку. Проблема была решена введением двух состояний завершения и отдельной проверкой режима после каждого вызова `Execute`. Для мягкой остановки используется завершение добавления элементов в `BlockingCollection`; перечисление существующих элементов при этом продолжается до конца.

Второе затруднение было связано с поведением пустой очереди. Цикл с постоянной проверкой `TryTake` работал бы даже при полном отсутствии полезной нагрузки и расходовал процессорное время пропорционально длительности простоя. В реализации применено блокирующее извлечение. Если элементов нет, системный поток переводится в состояние ожидания и возобновляется после добавления команды. Такой способ соответствует назначению серверного потока: ресурсы используются только при появлении работы, а задержка ожидания не создаёт активной нагрузки на процессор [10].

Исключение внутри одной команды не должно разрушать весь цикл обработки. Поэтому вызов `Execute` помещён в блок обработки исключений. При наличии `ExceptionHandler` в него передаются сама команда и полученное исключение, после чего сервер переходит к следующему элементу. Отдельными тестами проверялись оба вида остановки, запрет их внешнего вызова, обработка исключения, ожидание новой команды при пустой очереди и одновременная подача операций несколькими производителями. Наиболее сложным моментом задания оказалось согласование состояния очереди и режима завершения так, чтобы жёсткая остановка была немедленной, мягкая не теряла принятые операции, а пустой сервер не выполнял бесполезный цикл.

6.2 Реализация планировщика команд

Во втором задании потребовалось устранить монопольное использование потока длительной операцией. Если обычная команда выполняет всю работу за один вызов, сервер не может передать управление другим элементам до возврата из `Execute`. Для изменения этого поведения длительная операция была разделена на небольшие шаги. Интерфейс `ILongRunningCommand` расширяет `ICommand` свойством `IsCompleted`. Один вызов выполняет только очередной шаг, а признак завершения сообщает серверу, требуется ли планировать следующий вызов.

Планировщик определён интерфейсом `IScheduler`, который содержит операции проверки наличия работы, выбора следующей команды и добавления команды. Реализация `RoundRobinScheduler` хранит элементы в очереди. Выбранная команда удаляется из начала, выполняет один шаг и при незавершённом состоянии добавляется в конец. Благодаря этому при наличии команд *A*, *B* и *C* образуется последовательность *A, B, C, A, B, C, ...*. Каждая операция регулярно получает управление, поэтому одна длительная задача не вытесняет остальные.

Главная трудность возникла при совместном ожидании двух источников работы. Новые команды продолжают поступать во входную очередь, а ранее начатые длительные операции находятся в планировщике. Блокирующий вызов `Take` для входной очереди использовать нельзя: поток может заснуть при пустой входной очереди, хотя планировщик уже содержит незавершённую работу. Постоянный неблокирующий опрос обеих структур также неприемлем, поскольку при полном отсутствии команд он превращается в активный цикл.

Решение основано на сочетании `ConcurrentQueue<ICommand>` и `SemaphoreSlim` [16]. Из входной очереди элементы извлекаются неблокирующим способом. Если они отсутствуют, поток проверяет планировщик. Ожидание на семафоре начинается только тогда, когда оба источника пусты. Добавление новой команды увеличивает счётчик семафора и пробуждает обработчик. Повторное выполнение длительной операции производится через планировщик, а не через входную очередь, поэтому серверный поток не записывает в ту же очередь, из которой читает. Это устраняет описанный в задании риск мёртвой блокировки при ограниченной вместимости очереди.

Для сохранения отзывчивости поток чередует обработку нового элемента и шага запланированной операции. Несколько длительных команд, поступивших подряд, сначала регистрируются в планировщике, после чего выполняются циклически. Если во время их работы появляется обычная команда, она получает управление между двумя шагами длительной операции. Исключение в одном шаге передаётся обработчику и не приводит к бесконечному повторению неисправной команды. Команды `HardStopCommand` и `SoftStopCommand` сохраняют прежнюю семантику: жёсткая остановка прекращает цикл, а мягкая ожидает завершения входной очереди и всех элементов планировщика.

Набор тестов проверял порядок Round Robin, выбор из пустого планировщика, выполнение обычной команды до завершения длительной, поступление новой команды между двумя шагами, оба способа остановки, ожидание при полном отсутствии работы и обработку исключения. Наиболее сложной частью оказалось не само циклическое хранение элементов, а организация сна потока. Необходимо было исключить одновременно две противоположные ошибки: засыпание при наличии работы в планировщике и активный опрос при отсутствии любой работы.

6.3 Реализация длительных команд

В заключительном задании создан класс `TestCommand`. Он хранит идентификатор, счётчик вызовов и требуемое число шагов. После каждого вызова `Execute` счётчик увеличивается, а в поток вывода записывается строка вида «Поток 1 вызов 1». При достижении трёх вызовов свойство `IsCompleted` получает значение `true`. Сервер больше не возвращает такой экземпляр в планировщик. Дополнительный вызов завершённой команды не изменяет счётчик и не создаёт повторной строки.

Для демонстрации сформировано пять экземпляров с идентификаторами от 1 до 5. Все они добавляются в `ServerThread` и регистрируются планировщиком. Получается три полных цикла: в первом каждая команда вызывается один раз, во втором счётчики увеличиваются до двух, в третьем до трёх. Консольный проект `task19app` выводит ровно 15 строк. Тем самым длительные операции завершаются, но ни одна из них не удерживает серверный поток на протяжении всех трёх шагов.

Отдельная трудность была связана с требованием завершить демонстрацию именно командой `HardStop`. Если поместить обычную команду жёсткой остановки во входную очередь сразу после пяти длительных операций, сервер регистрирует длительные элементы в планировщике и выполнит остановку раньше их первого шага. Ожидание завершения в главном потоке с постоянной проверкой счётчиков, в свою очередь, повторило бы проблему активного опроса.

Для решения создана длительная команда `HardStopAfterCommand` `sCommand`. Она также участвует в Round Robin и после каждого цикла проверяет признаки завершения пяти рабочих команд. Пока хотя бы один элемент не завершён, команда остаётся в планировщике. После третьего цикла все признаки получают значение `true`; тогда команда выполняет `HardStop` непосредственно в серверном потоке. Проверка принадлежности потоку проходит успешно, и цикл прекращается. Такой вариант не требует ожидания со стороны консольного приложения и использует тот же механизм планирования, что и основные операции.

Тестами подтверждены три вызова одного экземпляра, отсутствие четвёртого вызова после завершения, поддержка другого числа шагов, проверка ошибочных параметров и полная демонстрация пяти команд. Интеграционный тест сравнивает весь вывод с ожидаемой последовательностью из 15 строк и убеждается, что сервер завершён жёсткой остановкой. Наиболее сложным оказался выбор момента `HardStop`: остановка должна выполняться не после фиксированной задержки, а после фактического завершения всех операций и только в целевом потоке.

6.4 Описание и интерпретация результатов экспериментов

Эксперименты выполнялись на той же реализации, которая использовалась в заданиях 17–19. Исходные результаты сохранены в файлах формата CSV. Первый опыт оценивал затраты процессорного времени при отсутствии команд. Для каждого варианта проводилось пять запусков длительностью одна секунда. В варианте с блокирующим ожиданием запускался пустой `ServerThread`; после одной секунды в него передавалась команда `HardStop`. Для сравнения выполнялся активный цикл `SpinWait` той же продолжительности. Измерялось изменение `Process.TotalProcessorTime`. Пе-

ред серией выполнялся короткий прогрев, уменьшающий влияние первичной компиляции методов.

Таблица 10: Процессорное время за одну секунду отсутствия полезной работы

Запуск	Блокирующее ожидание, мс	Активный опрос, мс
1	0,000	984,375
2	0,000	1000,000
3	0,000	984,375
4	0,000	1000,000
5	46,875	1000,000
Среднее	9,375	993,750



Рис. 3: Сравнение затрат процессорного времени при отсутствии команд

На рисунке 3 столбцы показывают средние значения, а отрезки погрешности соответствуют выборочному стандартному отклонению. Для блокирующего ожидания среднее процессорное время составило 9,375 мс, для активного опроса — 993,750 мс. Относительное уменьшение равно 99,06%. Нулевые значения в четырёх запусках означают, что затраты оказались ниже разрешения системного счётчика. Значение 46,875 мс в пятом запуске связано с кратковременной фоновой работой процесса и увеличивает стандартное отклонение, но не меняет общий вывод.

Полученная разница объясняется состояниями системного потока. При вызове `SemaphoreSlim.Wait` поток не получает кванты процессорного времени до появления сигнала. При активном опросе условие проверяется непрерывно, поэтому одно логическое ядро остаётся занятым почти всю секунду. Результат подтверждает выполнение требования об отсутствии значительных затрат, зависящих от продолжительности простоя. Небольшие постоянные расходы на запуск, пробуждение и остановку потока допустимы, поскольку они не растут пропорционально времени ожидания.

Во втором опыте фиксировался фактический порядок вызовов пяти экземпляров `TestCommand`. Данные получены из консольного вывода без перестановки строк. По оси абсцисс на рисунке 4 указан общий номер вызова `Execute`, по оси ординат — идентификатор выбранной команды.

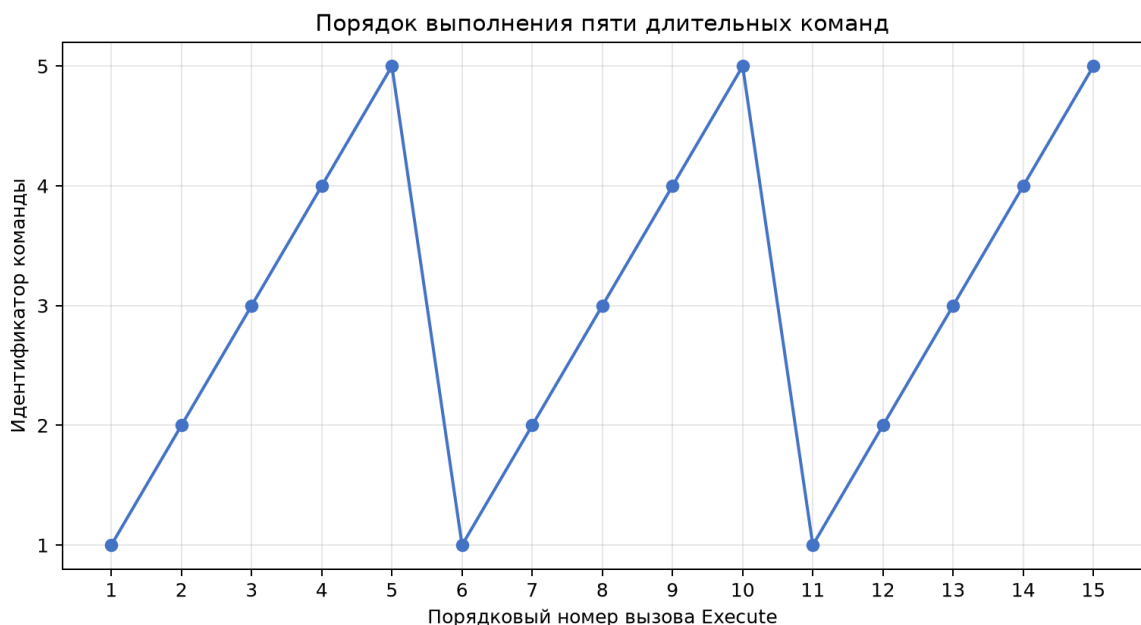


Рис. 4: Порядок выполнения пяти длительных команд планировщиком Round Robin

График состоит из трёх одинаковых циклов 1, 2, 3, 4, 5. После команды с идентификатором 5 планировщик возвращается к идентификатору 1. Ни одна команда не вызывается два раза подряд, пока другие готовые элементы ожидают выполнения. Расстояние между соседними вызовами одного экземпляра постоянно и равно пяти диспетчеризациям. Следовательно, стратегия обеспечивает слабую справедливость: при сохранении команды в планировщике она обязательно получает следующий шаг после конечного числа выборов.

Третий цикл заканчивается вызовом команды 5 с общим номером 15. К этому моменту счётчики всех пяти экземпляров равны трём. Следующая проверка `HardStopAfterCommandsCommand` обнаруживает завершение набора и останавливает сервер. На графике отсутствуют четвёртый цикл и преждевременное окончание, что одновременно подтверждает корректность признака `IsCompleted`, повторного добавления в конец очереди и условия жёсткой остановки. Форма графика определяется не продолжительностью отдельных шагов, а выбранной дисциплиной обслуживания: очередь `Round Robin` сохраняет порядок поступления и после каждого незавершённого шага переносит команду в конец.

СПИСОК ЛИТЕРАТУРЫ

- [1] Microsoft. Документация по C# [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 15.07.2026).
- [2] Microsoft. Модульное тестирование C# в .NET с помощью dotnet test и xUnit [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-csharp-with-xunit> (дата обращения: 15.07.2026).
- [3] xUnit.net. Официальная документация [Электронный ресурс]. URL: <https://xunit.net/> (дата обращения: 15.07.2026).
- [4] NUnit. Официальная документация [Электронный ресурс]. URL: <https://docs.nunit.org/> (дата обращения: 15.07.2026).
- [5] Microsoft. Рефлексия в .NET [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/fundamentals/reflection/reflection> (дата обращения: 15.07.2026).
- [6] Microsoft. Атрибуты и отражение в C# [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/advanced-topics/reflection-and-attributes/> (дата обращения: 15.07.2026).
- [7] Microsoft. Метод Assembly.LoadFrom [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.reflection.assembly.loadfrom> (дата обращения: 15.07.2026).
- [8] Microsoft. Пакет SDK для платформы компилятора .NET (API Roslyn) [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/roslyn-sdk/> (дата обращения: 15.07.2026).
- [9] Microsoft. Обзор System.Text.Json [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/overview> (дата обращения: 15.07.2026).
- [10] Microsoft. Использование потоков и работа с потоками [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/standard/threading/using-threads-and-threading> (дата обращения: 15.07.2026).

- [11] Microsoft. Примитив синхронизации Barrier [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/standard/threading/barrier> (дата обращения: 15.07.2026).
- [12] Microsoft. Класс Interlocked [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.threading.interlocked> (дата обращения: 15.07.2026).
- [13] Microsoft. Класс Stopwatch [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.diagnostics.stopwatch> (дата обращения: 15.07.2026).
- [14] ScottPlot. Quickstart for .NET Notebooks [Электронный ресурс]. URL: <https://scottplot.net/quickstart/notebook/> (дата обращения: 15.07.2026).
- [15] Microsoft. Потокбезопасные коллекции [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/standard/collections/thread-safe/> (дата обращения: 15.07.2026).
- [16] Microsoft. Класс SemaphoreSlim [Электронный ресурс]. URL: <https://learn.microsoft.com/ru-ru/dotnet/api/system.threading.semaphoreslim> (дата обращения: 15.07.2026).